Browser Forensics Analysis Tool

Summary

This project delivers a comprehensive browser forensics toolkit designed to automate the extraction and analysis of digital evidence from Chrome and Firefox browsers. The tool addresses critical gaps in digital forensics by providing investigators with an efficient, repeatable method for collecting browser artifacts. By automating data extraction from nine browser data sources and implementing intelligent heuristic detection for suspicious activity, this toolkit reduces investigation time, eliminating the need for manual analysis methods, while simultaneously improving accuracy and consistency.

1. Introduction

1.1 Purpose

Modern web browsers serve as the primary interface for communication, commerce, and work. This makes them central repositories of valuable digital forensic data. This project creates a unified forensics tool that systematically extracts and analyzes evidence from Chrome and Firefox browsers.

1.2 The Problem

Digital forensics investigators face several critical challenges when analyzing browser data. The first issue being manual extraction is time-intensive. Browser data is stored across multiple SQLite databases with complex schemas that differ between browsers and versions. Manually querying these databases for each data type can take hours per system and profile.The next issue is inconsistent methodologies. Without standardized tools, different investigators may use varying extraction techniques, leading to inconsistent results and potential gaps in evidence collection that could undermine the reliability of forensic findings. When it comes to digital forensics, it’s crucial that the evidence is irrefutable so that it may be held in a legal environment. After that, there comes the issue of timestamp complexity. Browsers store timestamps in different formats (Chrome uses WebKit time with microseconds since 1601, Firefox uses Unix microseconds since 1970). Converting these accurately while accounting for daylight saving time is error-prone when done manually. Another major issue is encryption barriers. Modern browsers encrypt sensitive data like saved passwords using platform-specific mechanisms (DPAPI on Windows, NSS library for Firefox), requiring specialized decryption techniques. Another key problem is database locking. Browsers maintain exclusive locks on SQLite databases while running, causing access errors unless proper workarounds are implemented. Finally would be the sheer volume of records and data to sort through. A typical

browser may contain thousands of history entries and hundreds of cookies. Identifying suspicious items within this volume requires automated heuristics.

1.3 The Solution

This project addresses these challenges by providing automated extraction of various artifact types, intelligent heuristic detection, accurate timestamp conversion with timezone awareness, integrated password decryption, safe database access through temporary file copying, and reporting in a singular comprehensive HTML output.

2. Technical Implementation

2.1 Architecture

The toolkit employs a three-layer architecture:

Layer 1: Specialized Extractors - Nine independent Python scripts handle specific artifact types:

- Chrome: History, Downloads, Extensions, Sessions, Autofill
- Firefox: History, Downloads, Extensions, Sessions

Each extractor is self-contained and can be executed independently for targeted analysis.

Layer 2: CLI - `run_all.py` coordinates execution:

- Manages subprocess execution of individual scripts
- Merges CSV outputs into unified datasets
- Applies filtering and correlation
- Generates HTML reports with interactive analysis
- Completed at the CLI for those who don't want to use GUI

Layer 3: Graphical User Interface - `run_all_gui.py` provides graphical access:

- Tkinter-based GUI with no external dependencies
- Interactive result exploration
- One-click HTML report viewing
- Output preview
- Easy argument selection

2.2 Programming Language and Core Libraries

Python 3.8+ was selected for cross-platform compatibility, library support, and cryptography libraries.

Standard Library Components:

- **`sqlite3`**
  - Direct access to browser SQLite databases
- **`csv` and `json`**
  - Structured output generation
- **`pathlib`**
  - Cross-platform file system operations
- **`datetime` and `zoneinfo`**
  - Timezone-aware timestamp conversion with automatic DST handling
- **`tempfile` and `shutil`**
  - Safe database copying to avoid lock conflicts
- **`tkinter`**
  - Zero-dependency GUI toolkit

External Libraries:

- **`cryptography`**
  - AES-GCM decryption of Chrome encrypted values
- **`pycryptodome`**
  - Alternative crypto implementation with fallback support
- **`pywin32` (`win32crypt`)**
  - Windows DPAPI access for Chrome key decryption
- **`python-lz4`**
  - Mozilla LZ4 decompression for Firefox session stores
- **`ctypes`**
  - NSS library interface for Firefox password decryption

2.3 Third-Party Code and AI Assistance

GitHub Repository Integration:

Two critical components were adapted from existing open-source projects:

1. **`Chrome_decrypt.py`**:
   Synthesized from community research on Chrome password decryption techniques, implementing DPAPI key unwrapping, AES-256-GCM decryption with version-prefixed ciphertext, and multiple key derivation fallback strategies.

2. **`firefox_decryptor.py`**:
   Derived from the firefox-decrypt project (https://github.com/unode/firefox_decrypt, GPL v3.0), adapted for programmatic library usage, non-fatal decryption mode, and multiple output formats.

AI-Assisted Development:

Claude (Anthropic's AI assistant) was utilized throughout development for:

- Code generation and refactoring
- Debugging assistance and crafting of SQLite schemas and timezone handling
- Documentation and comprehensive commenting
- Heuristic development for suspicious activity detection

This AI collaboration enabled rapid prototyping while maintaining forensic integrity standards.

2.4 Browser Data Extraction Techniques

Chrome Data Sources (profile directory: `%LOCALAPPDATA%\Google\Chrome\User Data\Default`):

- **History**
  - Query `History` SQLite database joining `urls` and `visits` tables; convert WebKit timestamps (microseconds since 1601-01-01 UTC)
- **Downloads**
  - Extract from `downloads` table with heuristics flagging executables, archives, and suspicious hosts
- **Extensions**
  - Parse `Extensions/` directory manifests and `Preferences` JSON; flag dangerous permissions (proxy, webRequest, cookies, nativeMessaging)
- **Sessions**
  - Sessionize visits within 30-minute timeout windows; correlate cookies and detect token leakage via regex patterns
- **Passwords**: Decrypt via DPAPI key extraction from `Local State`, then AES-256-GCM decrypt individual credentials

Firefox Data Sources (profile directory: `%APPDATA%\Mozilla\Firefox\Profiles\`):

- **History**
  - Query `places.sqlite` joining `moz_places` and `moz_historyvisits`; convert Unix microseconds
- **Downloads**
  - Primary extraction from `downloads.sqlite`; fallback to places.sqlite heuristics
- **Extensions**

    - Parse `extensions.json` and physical extension directories for permission analysis
- **Sessions**
    - Decompress `recovery.jsonlz4` (LZ4 with Mozilla magic header), correlate with cookies
- **Passwords**
    - NSS library integration via ctypes for credential decryption from `logins.json`

2.5 Timestamp Handling

Chrome WebKit Timestamps:

python
epoch = datetime(1601, 1, 1, tzinfo=timezone.utc)
dt_utc = epoch + timedelta(microseconds=chrome_time)

dt_local = dt_utc.astimezone(ZoneInfo("America/Chicago"))

Firefox Unix Timestamps:

python
dt_utc = datetime.fromtimestamp(firefox_time / 1_000_000, tz=timezone.utc)

dt_local = dt_utc.astimezone(ZoneInfo("America/Chicago"))

Implementation includes magnitude-based format detection (handles values >10^14 as WebKit, >10^11 as milliseconds, >10^8 as seconds) and automatic DST handling via zoneinfo/pytz with fallback warnings.

2.6 Heuristic Detection Systems

Extension Risk Assessment:

- Flags high-risk permissions: proxy, webRequest/Blocking, cookies, nativeMessaging, management
- `<all_urls>` host permission indicates content script injection on all websites
- Name-based keywords: "ad", "miner", "crypto", "proxy"

Cookie Security Analysis:

- `!isSecure`: HTTP transmission (interception risk)
- `!isHttpOnly`: JavaScript-accessible (XSS vulnerability)
- Expiry >365 days: Long-lived authentication tokens
- Third-party cookies: Domain mismatch suggests tracking
- Wildcard domains: Sent to all subdomains

Token Leakage Detection:

python:

TOKEN_RE =

re.compile(r"(token|access_token|auth|session|sessionid|sid|jwt)=([A-Za-z0-9_\-\.]{20,})")

Tokens in URLs leak through server logs, HTTP Referer headers, and browser history.

Download Risk Classification:

- Executables: .exe, .msi, .dll, .bat, .cmd, .scr
- Archives: .zip, .tar, .gz, .rar, .7z
- Cross-reference with Chrome danger_type field

2.7 Safe Database Access

Browsers lock SQLite databases during operation. Our implementation uses copy-before-read to ensure that this isn't an issue:

python
tmp = tempfile.NamedTemporaryFile(prefix='chrome_history_', delete=False)
shutil.copy2(str(src_path), tmp.name)

This approach preserves timestamps, avoids modifying evidence, enables analysis while browsers remain running, and automatically cleans up the temporary files after analysis.

3. Results and Forensic Value

3.1 Data Extraction Results

Browser Forensics Report

Generated: 2025-12-03T22:05:40.624359

Total rows: 2164 — suspicious: 223

Per-source summary

Source CSV	Rows	Suspicious
chrome_history.csv	1274	0
firefox_history.csv	377	0
chrome_sessions.csv	173	127
chrome_downloads.csv	76	12
firefox_sessions.csv	75	56
chrome_extensions.csv	63	23
chrome_autofill.csv	52	0
firefox_downloads.csv	50	5
firefox_extensions.csv	24	0

Discovered Firefox Passwords (per profile)

Profile: rcj1pid8.default-release (1 entries)

Website	Username	Password
https://www.example.com	sean	drsanders123

Discovered Chrome Passwords

Summary: 438 total password entries found

- **401** passwords protected by Chrome v127+ App-Bound Encryption (cannot be decrypted outside Chrome)
- **37** entries with no saved password

No Chrome passwords could be decrypted. All saved passwords are protected by Chrome's newer App-Bound Encryption feature (introduced in Chrome v127, July 2024), which prevents external tools from accessing them.

This screenshot shows the HTML forensic report that is created, it includes the per-source summary and the discovered passwords. The report also says how many items in each report are deemed “suspicious” according to some heuristics set up in our scripts.

All suspicious reasons (with explanations)

Reason	Count	Why it matters
token_in_url	127	Token or auth-like value found in URL — may leak via referer, logs, or history.
cookie_flags	56	Suspicious indicator; review context for details.
executable	17	Downloaded file is an executable — potentially dangerous if run.
permission: tabs	14	Requests tabs — can read/modify tab URLs and titles. (context: tabs)
permission: webRequest	10	Requests webRequest — can observe/modify network requests (powerful, privacy risk). (context: webRequest)
host:	7	Has host permission for — extension can access data on any website. (context:)
cookie_http_non_httponly: OTZ	6	Cookie is accessible to JavaScript (not HttpOnly) — increased XSS risk. (context: OTZ)
permission: cookies	6	Requests cookie access — can read and modify cookies, potentially hijacking sessions. (context: cookies)
permission: nativeMessaging	6	Requests nativeMessaging — can communicate with native apps on the host. (context: nativeMessaging)
cookie_http_non_httponly: OptanonConsent	5	Cookie is accessible to JavaScript (not HttpOnly) — increased XSS risk. (context: OptanonConsent)
cookie_insecure: OptanonConsent	5	Cookie not marked Secure — transmitted over HTTP and may be intercepted. (context: OptanonConsent)
cookie_http_non_httponly: itua	4	Cookie is accessible to JavaScript (not HttpOnly) — increased XSS risk. (context: itua)
cookie_http_non_httponly: media-user-token	4	Cookie is accessible to JavaScript (not HttpOnly) — increased XSS risk. (context: media-user-token)
cookie_http_non_httponly: mut-refresh	4	Cookie is accessible to JavaScript (not HttpOnly) — increased XSS risk. (context: mut-refresh)
cookie_http_non_httponly: pldfltcid	4	Cookie is accessible to JavaScript (not HttpOnly) — increased XSS risk. (context: pldfltcid)
cookie_http_non_httponly: pltvcid	4	Cookie is accessible to JavaScript (not HttpOnly) — increased XSS risk. (context: pltvcid)
cookie_insecure: itua	4	Cookie not marked Secure — transmitted over HTTP and may be intercepted. (context: itua)
cookie_insecure: pldfltcid	4	Cookie not marked Secure — transmitted over HTTP and may be intercepted. (context: pldfltcid)
cookie_insecure: pltvcid	4	Cookie not marked Secure — transmitted over HTTP and may be intercepted. (context: pltvcid)
cookie_http_non_httponly: NetworkProbeLimit	3	Cookie is accessible to JavaScript (not HttpOnly) — increased XSS risk. (context: NetworkProbeLimit)
cookie_http_non_httponly: OptanonAlertBoxClosed	3	Cookie is accessible to JavaScript (not HttpOnly) — increased XSS risk. (context: OptanonAlertBoxClosed)

This screenshot is from the HTML report and shows the reasons for the different suspicious items.

Suspicious Extensions Details

chrome_extensions.csv

Profile	Extension ID	Name	Reasons (with explanation)
Default	inomeogfingihgjfjlpeplalcfajhgai	Chrome Remote Desktop	permission:nativeMessaging — Requests nativeMessaging — can communicate with native apps on the host. permission:downloads — Requests downloads permission — can manage or read downloads.
Profile 1	bkdgflcldnnnapblkhphbgpggdiikppg	__MSG_appName__	permission:tabs — Requests tabs — can read/modify tab URLs and titles. permission:webRequest — Requests webRequest — can observe/modify network requests (powerful, privacy risk). permission:cookies — Requests cookie access — can read and modify cookies, potentially hijacking sessions.
Profile 1	bkkjeefjfjcfdfifddmkdmcpmaakmelp	Truffle	permission:cookies — Requests cookie access — can read and modify cookies, potentially hijacking sessions.
Profile 1	bmnlcjabgnpnenekpadlanbbkooimhnj	__MSG_Honey_Title__	permission:cookies — Requests cookie access — can read and modify cookies, potentially hijacking sessions. permission:webRequest — Requests webRequest — can observe/modify network requests (powerful, privacy risk).
Profile 1	cfhdojbkjhnklbpkdaibdccddilifddb	__MSG_name_releasebuild__	permission:tabs — Requests tabs — can read/modify tab URLs and titles. permission:webRequest — Requests webRequest — can observe/modify network requests (powerful, privacy risk). host: — Has host permission for — extension can access data on any website.
Profile 1	chphlpgkkbolifaimnlloiipkdnihall	OneTab	permission:tabs — Requests tabs — can read/modify tab URLs and titles.
Profile 1	cjpalhdlnbpafiamejdnhcphjbkeiagm	uBlock Origin	permission:tabs — Requests tabs — can read/modify tab URLs and titles. permission:webRequest — Requests webRequest — can observe/modify network requests (powerful, privacy risk). permission:webRequestBlocking — Requests webRequestBlocking — can synchronously modify or block requests.
Profile 1	ddkjiahejlhfcafbddmgiahcphecmpfh	__MSG_extName__	host: — Has host permission for — extension can access data on any website.
Profile 1	dmgkiikdlhmpikkhpiplldicbnicmboc	__MSG_appName__	host: — Has host permission for — extension can access data on any website.
Profile 1	emhginjpijfggbofeediiojmdlmlkoik	Avast Passwords	permission:tabs — Requests tabs — can read/modify tab URLs and titles. host: — Has host permission for — extension can access data on any website.

This screenshot is from the HTML report and shows some of the details for the extensions that are deemed “suspicious” and it explains what permissions make it suspicious.

3.2 Forensic Value Delivered

- **Timeline Reconstruction**
  - Precise chronological ordering enables correlation with system activity. Timezone-aware timestamps ensure accurate sequencing so that forensic investigators are able to put together events in the order of which they occurred.
- **Credential Compromise Assessment**
  - Decrypted passwords verify breach exposure; correlation identifies credential reuse patterns.
- **Malware Behavior Evidence**
  - Download history reveals infection vectors; extension analysis detects browser-based malware and cryptominers.
- **Insider Threat Detection**

    - History identifies visits to competitor sites or file sharing services; download records show potential data exfiltration.
- **Incident Response Support**
    - Rapid extraction allows for a quick initial assessment. Our automated flagging also helps prioritize investigation areas so the crucial issues can be addressed first.

3.3 Comparison to Manual Analysis

- **Time Savings**
    - Manual extraction can take a lot of time depending on how many databases you intend on analyzing. Having automated scripts that can pull all of the information immediately saves countless hours.
- **Completeness**
    - Manual analysis often misses secondary sources. Automated analysis guarantees consistent coverage of all eight artifact types.
- **Accuracy**
    - Automated timestamp conversion eliminates human error in timezone calculations and DST handling.
- **Scalability**
    - Manual analysis by nature is much slower than automated analysis, allowing for a much larger scale of systems to be analyzed on a day to day basis.

---

4. Conclusion and Future Work

4.1 Achievements

Our project successfully delivers a production-ready browser forensics toolkit with comprehensive data coverage across eleven artifact types, a forensically sound methodology with non-destructive analysis, and accessible interfaces for diverse skill levels, ensuring anyone is able to use our tool.

4.2 Problems encountered

The main problem we encountered while crafting this toolkit was getting the chrome_decrypt.py script to function correctly. As it stands now, the script is unable to process the login information and decrypt the passwords. After researching into the issue, we found that after July of 2024, Chrome appears to have updated their encryption methods and made it extremely difficult to decrypt the information in the login database file.

Another problem encountered was getting the GUI to function as we wanted it to. The output files were unable to be opened through the GUI at first, you had to open the File Explorer

instead. It was also difficult getting the output files to preview in our GUI when hovering over them.

Lastly, we believe that we were a little over ambitious when it came to the features that we wanted to add to this project. As it stands, the scripts support Chrome and Firefox on both Windows and Mac operating systems. We would've liked to have added Linux support, as well as other browser support, but we didn't have enough time. We don't believe that adding support for other Chromium-based browsers would have taken too much time, as the only real difference would most likely have been the file path to which the database files are located, and a difference in the encryption methods for the logins.

4.3 Future Enhancements

- **Extended Browser Support**
  - Edge, Safari, Brave, Vivaldi, Opera
- **Advanced Analytics**
  - Machine learning anomaly detection, behavioral profiling, entity extraction, link analysis
- **Enhanced Visualization**
  - Interactive timelines, network graphs, geolocation mapping, heat maps
- **Enterprise Integration**
  - SIEM integration, scheduled execution, multi-system dashboards, threat intelligence feeds
- **Mobile Support**
  - Android Chrome and iOS Safari extraction from device backups

4.4 Educational Impact

Our project demonstrates practical database forensics, real-world cryptography, software engineering best practices, and digital forensics methodologies. It provides hands-on experience with forensic tool development, reverse engineering, cross-platform design, and technical documentation while also contributing to the open-source forensics community.

References

Browser Documentation: Chromium Project, Mozilla Developer Network, SQLite Documentation

Forensic Standards: NIST SP 800-86, SANS DFIR resources, OWASP Browser Security

Open Source: firefox-decrypt (unode, GPL v3.0), cryptography library (BSD), pycryptodome (BSD), python-lz4 (BSD)

AI Assistance: Claude (Anthropic) for code generation, debugging, and documentation